

Relazione Progetto Lab III CONNECTIONS:

Collega le parole e vinci!

Il progetto implementa il gioco Connections seguendo un'architettura Client-Server in Java. Il sistema si basa su una singola partita globale a cui partecipano in tempo reale tutti i giocatori loggati, i quali condividono una sessione di gioco a tempo.

1. Scelte effettuate nei punti lasciati a libera interpretazione

1. Messaggi asincroni UDP per notifica fine partita

Per l'inoltro di pacchetti UDP ho utilizzato una registrazione dinamica. All'avvio il client apre il suo DatagramChannel effettuando bind sulla porta 0 (`udpChannel.bind(new InetSocketAddress(0))`). Questa istruzione ordina all'OS di associare al canale una porta libera a sua scelta. Il client poi recupera tale porta tramite `client.getUdpPort()`. Quando poi effettua il login, il client include automaticamente il valore della porta UDP nella richiesta Json (`req.udpPort = client.getUdpPort()`).

All'arrivo della richiesta di login sul server, esso estrae l'IP remoto del client direttamente dal socket TCP e lo combina con la porta UDP trasmessa. In questo modo il server può inviare datagrammi di broadcast senza rischiare conflitti di porte dove un host connette più client.

2. Caricamento in stream del dataset delle parole

Nel progetto viene richiesto di trattare il file delle partite `word.json` come se avesse grandi dimensioni, il che esclude che si possa caricare l'intero Json in memoria all'avvio. Questo problema viene risolto con la classe `WordDataset.java` implementando lo streaming di token con la classe `JsonReader` di Gson.

Il programma effettua il conteggio delle partite con la funzione `countGames` dove il file viene scorso sequenzialmente per token. Quando poi `GameManager` deve avviare una nuova partita casuale genera un indice target. `WordDataset` riapre il file in modalità stream e salta fino al valore scelto casualmente (si cerca di evitare partite già giocate).

3. Controllo proposte mal formate

Nel progetto si specifica come le proposte mal formate non debbano influire sul punteggio. In `handleSubmitProposal` si cerca di evitare tutti i casi. Si controlla se ci sono parole duplicate, che le parole in una proposta non siano già presenti in gruppi già indovinati e che soprattutto le parole appartengano alle 16 parole della partita in corso.

4. Spengimento controllato

Per azzerare il rischio di perdita di dati negli ultimi minuti di attività, prima di un riavvio volontario o forzato del server (ad esempio con un SIGINT) il ServerMain utilizza un thread di spengimento di emergenza il quale utilizza lo stesso meccanismo dello StorageManager per il salvataggio di dati sul disco.

2. Schema generale dei Thread attivati (server)

1. Main thread:

Esegue il metodo start() in NioServer.java. È un thread non-bloccante che esegue continuamente il ciclo selector.select(). Accetta le nuove connessioni TCP (ServerSocketChannel), legge i pacchetti di byte in ingresso dai socket TCP dei clienti attivi e infine delega l'elaborazione dei messaggi al Thread Pool

2. Thread Pool:

Viene istanziato con Executors.newCachedThreadPool(). Ho scelto un pool dinamico non sapendo bene l'architettura della macchina su cui ipoteticamente girerei il server (sulla mia macchina non ho notato differenza con un Fixed con numero di core). Ogni thread worker esegue il parsing del messaggio JSON, invoca il controllore logico CommandProcessor.process(...) per validare le richieste ed effettuare gli eventuali cambiamenti di stato e riceve una risposta JsonResponse che scrive direttamente sul SocketChannel del client.

3. GameManager:

Un ScheduledExecutorService a singolo thread (newSingleThreadScheduledExecutor) dedicato alla partita globale. Gestisce il timer del gioco. Si mette a dormire per la durata configurata del match e allo scadere esegue StartNextGame() che congela lo stato dei partecipanti e carica la nuova partita. Infine invia i datagrammi UDP e avvia il thread di notifica TCP.

4. StorageManager:

Un ScheduledExecutorService a singolo thread dedicato al salvataggio periodico. Si sveglia ad intervalli regolari per scrivere in modo su disco lo stato degli utenti in user.json e i giochi passati in games.json.

3. Schema generale dei Thread attivati (client)

1. **Main thread:** Gestisce l'interfaccia da CLI. Rimane costantemente bloccato in attesa di un comando su `scanner.nextLine()`. Infine all'inserimento di un comando crea un oggetto `JsonRequest` e chiama `client.sendRequest(req)`.
2. **Network thread:**
Implementato con l'interfaccia `Runnable`, utilizza un `Selector` per monitorare contemporaneamente due canali non bloccanti (`tcpChannel` e `udpChannel`). Quando riceve dati da uno dei due canali il thread di rete si sveglia, parse l'evento `Json` e lo formatta a schermo.

4. Strutture dati utilizzate

1. **RegisteredUsers:**
`private final ConcurrentHashMap<String, User> registeredUsers = new ConcurrentHashMap<>();`
È una `ConcurrentHashMap` ed è la struttura dati centrale del server. Memorizza tutti i profili utente registrati, utilizzando il nome utente come chiave (case sensitive) e l'oggetto `User` come valore. Essendo `concurrent` garantisce la sicurezza nei thread per letture e scritture simultanee.
2. **ActiveConnections:**
`private final ConcurrentHashMap<SocketChannel, User> activeConnections = new ConcurrentHashMap<>();`
È una `ConcurrentHashMap` e consente al server di tracciare in tempo reale quali utenti sono attualmente collegati e su quali canali fisici. L'associazione viene creata al login. Alla disconnessione il socket viene rimosso in modo pulito.
3. **UserUdpAddresses:**
`private final ConcurrentHashMap<String, InetSocketAddress> userUdpAddresses = new ConcurrentHashMap<>();`
Associa il nome utente loggato all'indirizzo IP remoto e alla porta UDP.
4. **pastGames:**
`private final Map<Integer, HistoricalGameStats> pastGames = new ConcurrentHashMap<>();`
Memorizza i metadati e le statistiche aggregate di tutte le partite concluse nel tempo. Popolata ad ogni scadenza del timer.
5. **CurrentGame**
`private final AtomicReference<Game> currentGame = new AtomicReference<>();`
Riferimento che punta all'oggetto `Game` al momento attivo su server.

5. Primitive di Sincronizzazione per l'Accesso alle Strutture Condivise

Nel progetto viene utilizzata la parola chiave “synchronized” in 3 contesti strategici:

1. Registrazione e aggiornamento di credenziali:
Durante l’inserimento di un nuovo utente o la modifica del nome utente due thread worker potrebbero verificare che il nome utente sia libero nello stesso modo o modificare la password contemporaneamente e invalidare il valore.
2. Invio di proposte simultanee:
Se un utente inviasse rapidamente 2 risposte identiche 2 thread di si troverebbero a modificare contemporaneamente le variabili di stato `currentMistak` e `currentScore` dell’oggetto `User`. Per questo motivo si utilizza `synchronized (user)`.
3. Transizione dello stato di gioco:
`ResetGameState` è dichiarato `synchronized` per garantire che il consolidamento dello stato temporaneo nelle statistiche storiche definitive dell’utente avvenga in modo atomico, impedendo a `StorageManager` di leggere un record utente parzialmente aggiornato.

6. Spiegazione della timeline del Main

1. Avvio il main (MAIN).
2. Il Main chiama `ConfigReader` passando il path del file delle server properties (MAIN).
3. `ConfigReader` usa un oggetto `Properties` per caricare tutte le proprietà con chiave-valore. (CONFIGREADER)
4. Il main fa `.getIntProperty` e `.getProperty` per prendere i valori di config (MAIN).
5. Il main inizializza il `NioServer` per ascoltare le connessioni (MAIN).
6. `NioServer` inizializza con la porta passata dal main e crea una `CachedThreadPool` e un `Gson` per leggere le stringhe json (NIOSEVER).
7. Il Main inizializza un `GameManager` passando il percorso delle parole, il tempo di una parita e il `serverNio` (MAIN).
8. `GameManager` stanza i parametri e inizializza un `ScheduledThread` per gestire la fine della partita (GAMEMANGER).
9. Il `GameManager` conta le partite su file, e inizia una nuova partita scegliendone una causale. Poi imposta un thread schedulato. Il thread addormentato si sveglia e: Manda messaggio di fine sul terminale del server. Chiama `startNextGame()` la quale salva i valori della partita nello storico e usa `User.resetGameState(-1)` per salvare i dati dell'utente. Poi `startNextGame()` chiama `server.broadcastNewGameStart(newGame)` il quale schedula un thread tra un secondo per scrivere (nuova partita iniziata) e poi ritorna. `StartNextGame` nel frattempo schedula se stessa un altro thread con il timer della partita che rifarà la stessa cosa. Il thread risvegliato infine usa `broadcastGameEnd` per annunciare la fine della partita al client e stampare il resoconto della partita (con notifica asincrona UDP) e nel frattempo il thread schedulato tra un secondo finisce e manda il messaggio di nuovo inizio partita. Il thread risvegliato termina e ricomincia il ciclo quando si sveglia l'altro thread schedulato con il timer della partita. (GAMEMANAGER)
10. Il main dunque prende i valori della cartella con le parole e il timer di intervallo dei salvataggi automatici. (MAIN)
11. Il main passa quei valori allo `StorageManager` + la lista degli utenti (MAIN)
12. Lo `StorageManager` inizializza un thread schedulato (STORAGEMANAGER)
13. Il main dice allo `StorageManager` di caricare tutti gli utenti da fine nella mappa condivisa `registeredUsers`. (MAIN-STORAGEMANAGER)

14. Faccio lo stesso con le partite precedenti (MAIN-STORAGEMANAGER)
15. Dunque imposto il thread schedulato di StorageManager in quale ogni intervallo chiama le due funzioni saveUsers e savePastGames le quali scrivono su file gli utenti e le partite precedenti dalla memoria (MAIN-STORAGEMANAGER)
16. Imposto la cattura del SIGINT il quale chiama la funzione di salvataggio di StorageManager ed esce (MAIN)
17. Avvio il GameManager che fa quello che è scritto sopra (MAIN)
18. Avvio il NioServer (MAIN)
19. Il NioServer crea un selector, poi si mette in ascolto sulla porta passata. Poi fa il classico ciclo con i selector dove aspetta le richieste di connessione. Quando riceve una richiesta di connessione chiama acceptConnection la quale tira fuori il canale di ascolto, fa una .accept() su di esso e ricava un oggetto SocketChannel chiamato client. Dopo di che si mette in modalità non bloccante e registra il nuovo channel per eventi di lettura (quando scriverà).
20. Per la lettura tira fuori il canale del client. Poi tira fuori il buffer collegato alla chiave e usa clientChannel.read() per scriverci dentro. Se legge -1 vuol dire che ha mandato la chiusura e verrà gestito dall'altra funzione. A questo punto fa la flip sul buffer e memorizza il messaggio in un array di byte (per la scrittura). Genero una stringa dal quel vettore di byte e uso il ThreaPool per mandare la stringa ad elaborare con la funzione processMessage. (NioServer)
21. ProcessManage usa il messaggio ricevuto e con la funzione .fromJson lo converte in un oggetto JsonRequest. A questo punto manda questo oggetto e il canale del client alla funzione CommandProcessor.process(...), la quale restituirà un oggetto di tipo JsonResponse. Questo oggetto infine sarà inviato di nuovo sul canale del client con sendResponse(clientChannel, ...) insieme al riferimento al server stesso. (NioServer)
22. In CommandProcessor arriva l'oggetto richiesta JsonRequest, il canale SocketChannel e il server. CommandProcessor smista il messaggio basandosi sul campo "operation" della richiesta e esegue una funzione di handle rispettivamente. Per operazioni che richiedono modifiche ai valori di utenti (come registrare un utente, aggiornare le credenziali) sono fatte con blocchi di sincronizzazione. Le JsonResponse possono essere success o error distinte dal campo "status". È poi presente un campo (opzionale) di codice di errore e di messaggio di errore. Infine è presente un campo di payload opzionale "data" che contiene dati (solo se success) in risposta ad una richiesta del client

6. Compilazione ed esecuzione

Per compilare il progetto basta eseguire il seguente comando:

```
javac -cp "lib/gson-2.10.1.jar" -d out src/common/*.java src/server/*.java src/client/*.java
```

Per l'esecuzione di Server e Client sono relativamente:

```
java -cp "out:lib/gson-2.10.1.jar" server.ServerMain
```

```
java -cp "out:lib/gson-2.10.1.jar" client.ClientMain
```

È inclusa nel progetto la libreria Gson sotto forma di Jar. I comandi eseguibili sono tutti quelli definiti nel pdf del progetto.